Contents

1 Introduction to R

In PS0001, you learnt Python and its syntax. In this module, we use another language, R. Generally speaking, R was built for statistical analysis, with many built-in functions. Similar functionality in Python must be imported from another library.

Why use R over Python? Well, it's just a different language.

Many of the same concepts overlap - variables, functions, iteration, etc. However, a slight difference is that unlike in PS0001, we will actually be much more hands-on with the running of our code - we are interested in running a function, and seeing its output, immediately.

2 The `cat()` and `print()` functions

The `cat()` function (short for concatenate) and the `print()` function can be used to display information to the user. Both, however, work slightly differently.

We will not frequently use the `cat()` or `print()` functions, because we generally see the output immediately.

```
cat('abc')
print('xyz')
```

3 Assignment operator (<- or =)

<- and = can both be used to assign values to variables in R. For consistency I will generally use <-.

```
x = 1
y <- 2
cat('x has the value', x, ', y has the value', y)
```

```
## x has the value 1 , y has the value 2
```

4 Vectors (c())

Vectors are created using the `c()` function. When a range of consecutive numbers is desired, we can also use `start:end` to get a vector of all numbers between `start` and end, inclusive of both ends.

```
v1 <- c(1, 3, 5, 7, 9, 11)
v2 <- 1:6*2   # create the vector (1, 2, 3, 4, 5, 6) and multiply each
↪  element by 2.
cat('v1 has the value', v1, ', v2 has the value', v2)
```

```
## v1 has the value 1 3 5 7 9 11 , v2 has the value 2 4 6 8 10 12
```

You can see the length of a vector using `length()`:

```
cat('v1 has length', length(v1))
```

```
## v1 has length 6
```

5 Data frames (`data.frame`)

Data frames are what we will be handling throughout the rest of the course. They are a collection of data, organised in **rows** and **columns**. They can be created with the `data.frame()` function. In this case, there are 3 sets of data.

```
df <- data.frame(c(26, 24), c(175, 172), c("M", "F"))
df
```

```
## # A tibble: 2 x 3
##   c.26..24. c.175..172. c..M....F..
##       <dbl>       <dbl> <chr>
## 1        26         175 M
## 2        24         172 F
```

The rows are individual observations, the columns are the data of each observation. By default, the columns are not assigned any meaningful names, and the rows are assigned numerical labels. We can see the column names and assign new names using the `names()` (or `colnames()` function.

```
names(df) # equivalently, colnames(df)
```

```
## [1] "c.26..24."   "c.175..172." "c..M....F.."
```

```
names(df) <- c('age', 'height', 'gender')
df
```

```
## # A tibble: 2 x 3
##     age height gender
##   <dbl>  <dbl> <chr>
## 1    26    175 M
## 2    24    172 F
```

The column names could also be given directly in the `data.frame()` function:

```
df1 <- data.frame(age = c(26, 24), height = c(175, 172), gender = c("M",
↪  "F"))
df1
```

```
## # A tibble: 2 x 3
##     age height gender
##   <dbl>  <dbl> <chr>
## 1    26    175 M
## 2    24    172 F
```

We can also change the row names if desired using `rownames()`.

```
rownames(df)
```

```
## [1] "1" "2"
```

```
rownames(df) <- c('caleb', 'taylor')
df
```

```
## # A tibble: 2 x 3
##     age height gender
##   <dbl>  <dbl> <chr>
## 1    26    175 M
## 2    24    172 F
```

5.1 Dimension of a data frame (or matrix) (`dim()`)

`dim()` tells us the dimension of an object. A vector has no dimension, only length. In this case, `dim()` tells us there are 2 rows and 3 columns.

```
dim(df)
```

```
## [1] 2 3
```

Now, to describe this data frame, I will say that this data frame has 2 observations (rows), and 3 data points (columns) for each observation. The interpretation of this data frame with these row and column names should come naturally based on the column and row names.

In this course, in general, we will not be creating data frames directly. Instead, most data will be imported using a `read` function, which we will see in the lab, or imported from another library, which we will see in a few weeks.

5.2 Getting help (? and `help()`)

Many functions in R have documentation on them, i.e. what the function does, how the function is used, the parameters it accepts. If you ever forget what a function does or how to use it, in the R console you can use the ? operator or the `help()` function.

For example, typing `?names` or `help(names)` *without the parentheses* will bring up the documentation page for the `names()` function. You can do this with nearly anything, e.g `c()`, `dim()`, `data.frame()`

6 Indexing vectors, matrices, and data frames

To get a single value in a vector, matrix, or data frame, we can use indexing with square brackets `[]`. Indexing in R is extremely similar to indexing in Python, but unlike Python, which is 0-indexed, R is 1-indexed, so the first entry in a list always has index 1. Vectors in R are analogous to lists in Python (that do not contain other lists), and require only one indexing.

```
x <- 1:5*10
x
```

```
## [1] 10 20 30 40 50
```

```
x[2]            # Get the second element
```

```
## [1] 20
```

For matrices and data frames, as they have two dimensions (rows and columns), we have the option to get rows, columns, or individual cells. The square bracket notation is the same, but we now have the option to index along the rows or columns or both. Take note of the below examples, noting the presence of the commas and blank entries.

```
df['age']     # with no comma, selects columns, return as *data frame*.
```

```
## # A tibble: 2 x 1
##     age
##   <dbl>
## 1    26
## 2    24
```

```
              # or with comma, first is row, second is column.
df[, 'age']   # selects all rows, and only column 'age', return as
↪  *vector*.
```

```
## [1] 26 24
```

```
df['caleb',]  # selects only row 'caleb', and all columns, return as *data
↪  frame*.
```

```
## # A tibble: 1 x 3
##     age height gender
##   <dbl>  <dbl> <chr>
## 1    26    175 M
```

Take note of the difference with and without the comma. Without the comma, it is returned as a **data frame**. With the comma, it is returned as a **vector**. This distinction will not be useful for a while, but some functions accept **only vectors** or **only data frames**.

In general, the indexing is done as `thing_to_index[rows, columns]`. Named columns in data frames can also be accessed using a `$` operator.

```
df$age          # equivalent to df2[, 'age'], return as vector.
```

```
## [1] 26 24
```

To select multiple rows or multiple columns, you can pass in vectors as indexes. Selections can also be made using numerical indices. You can see that the below have the exact same output as using the named indices above.

```
df
```

```
## # A tibble: 2 x 3
##     age height gender
##   <dbl>  <dbl> <chr>
## 1    26    175 M
## 2    24    172 F
```

```
df[1:2]         # get the first to second columns
```

```
## # A tibble: 2 x 2
##     age height
##   <dbl>  <dbl>
## 1    26    175
## 2    24    172
```

```r
df[1, ]        # get the first row, and all columns.
```

```
## # A tibble: 1 x 3
##     age height gender
##   <dbl>  <dbl> <chr>
## 1    26    175 M
```

```r
df[, c(1, 3)]  # get all rows, and the first and third columns
```

```
## # A tibble: 2 x 2
##     age gender
##   <dbl> <chr>
## 1    26 M
## 2    24 F
```

Indexing for vectors and matrices is extremely similar. As an exercise, index the following values from the below matrix with a **single indexing operation**, using the c ( ) function if necessary.

```r
m1 <- matrix(1:20, nrow=4, byrow=T)
m1
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    2    3    4    5
## [2,]    6    7    8    9   10
## [3,]   11   12   13   14   15
## [4,]   16   17   18   19   20
```

```r
# Index the following from the matrix m1:
# The entry 13
# The entries 1, 2, 3, 4, and 5.
# The entries 4, 9, 14, and 19.
# The entries 2, 7.
# The entries 1, 3, 16, and 18.

# For example, to index the entry 12
# which is the 3rd row, 2nd column.
m1[3, 2]
```

```
## [1] 12
```

```
m1[3,3]
```

```
## [1] 13
```

```
m1[1,]
```

```
## [1] 1 2 3 4 5
```

```
m1[,4]
```

```
## [1]  4  9 14 19
```

```
m1[c(1,2),2]
```

```
## [1] 2 7
```

```
m1[c(1,4),c(1,3)]
```

```
##      [,1] [,2]
## [1,]    1    3
## [2,]   16   18
```